

retail_Sliver_layer:

```
1  AIRBNSRETAIL.BRONZE.CUSTOMER_STAGE-----silver Layer-----
2  use schema silver;
3
4  -- Set correct role and database
5  USE ROLE ACCOUNTADMIN;
6  USE DATABASE RETAIL;
7
8  -- Create SILVER schema if it does not exist
9  CREATE SCHEMA IF NOT EXISTS RETAIL.SILVER;
10
11 -- Create Silver products table
12 CREATE OR REPLACE TABLE RETAIL.SILVER.PRODUCTS AS
13 SELECT DISTINCT
14     product_id,
15     TRIM(product_name) AS product_name,
16     TRIM(category) AS category,
17     TRY_TO_NUMBER(price) AS price,
18     CURRENT_TIMESTAMP() AS processed_ts
19 FROM RETAIL.BRONZE.PRODUCTS
20 WHERE product_id IS NOT NULL;
21
22 select * from RETAIL.SILVER.PRODUCTS;
23
24
25 CREATE OR REPLACE TABLE retail.silver.customers AS
26 SELECT
27     customer_id,
28     TRIM(customer_name) AS customer_name,
29     TRIM(gender) AS gender,
30     TRY_TO_NUMBER(age) AS age,
31     TRIM(city) AS city,
32     CURRENT_TIMESTAMP() AS processed_ts
33 FROM retail.bronze.customers
34 QUALIFY ROW_NUMBER() OVER (
35     PARTITION BY customer_id
36     ORDER BY customer_id
37 ) = 1;
38
39
40 CREATE OR REPLACE TABLE retail.silver.sales AS
41 SELECT
42     customer_id,
43     product_id,
44     store_id,
45     TRY_TO_NUMBER(quantity) AS quantity,
46     order_date, -- already TIMESTAMP
47     COALESCE(discount_pct, 0) AS discount_pct,
48     CURRENT_TIMESTAMP() AS processed_ts
49 FROM retail.bronze.raw_sales
50 WHERE TRY_TO_NUMBER(quantity) IS NOT NULL;
51
52 CREATE OR REPLACE TABLE retail.silver.stores AS
53 SELECT
54     store_id,
55     TRIM(store_name) AS store_name,
56     CURRENT_TIMESTAMP() AS processed_ts
57 FROM retail.bronze.stores
58 WHERE store_id IS NOT NULL;
59
60
```

Gold_layer:

```
-- -----gold Layer-----
61
62 CREATE OR REPLACE TABLE retail.gold.product_dim AS
63 SELECT
64     ROW_NUMBER() OVER (ORDER BY product_id) AS product_key,
65     product_id,
66     product_name,
67     category,
68     price
69 FROM retail.silver.products;
70
71 CREATE OR REPLACE TABLE retail.gold.store_dim AS
72 SELECT
73     ROW_NUMBER() OVER (ORDER BY store_id) AS store_key,
74     store_id,
75     store_name
76 FROM retail.silver.stores;
77
78 CREATE OR REPLACE TABLE retail.gold.customer_dim AS
79 SELECT
80     ROW_NUMBER() OVER (ORDER BY customer_id) AS customer_key,
81     customer_id,
82     customer_name,
83     gender,
84     age,
85     city
86 FROM retail.silver.customers;
87
88 CREATE OR REPLACE TABLE retail.gold.date_dim AS
89 SELECT DISTINCT
90     order_date AS date_key,
91     YEAR(order_date) AS year,
92     MONTH(order_date) AS month,
93     DAY(order_date) AS day
94 FROM retail.silver.sales;
95
96
```

```

88
89 CREATE OR REPLACE TABLE retail.gold.date_dim AS
90 SELECT DISTINCT
91     order_date                AS date_key,
92     YEAR(order_date)          AS year,
93     MONTH(order_date)         AS month,
94     DAY(order_date)           AS day
95 FROM retail.silver.sales;
96
97 CREATE OR REPLACE TABLE retail.gold.sales_fact AS
98 SELECT
99     st.store_key,
100     pr.product_key,
101     cu.customer_key,
102     dt.date_key,
103     COALESCE(sl.quantity, 0) AS quantity,
104     COALESCE(sl.discount_pct, 0) AS discount_pct
105 FROM retail.gold.store_dim st
106 LEFT JOIN retail.silver.sales sl
107     ON st.store_id = sl.store_id
108 LEFT JOIN retail.gold.product_dim pr
109     ON sl.product_id = pr.product_id
110 LEFT JOIN retail.gold.customer_dim cu
111     ON sl.customer_id = cu.customer_id
112 LEFT JOIN retail.gold.date_dim dt
113     ON DATE(sl.order_date) = dt.date_key;
114
115

```

Kpi:

```

117
118 CREATE OR REPLACE DYNAMIC TABLE retail.gold.dt_total_sales
119 WAREHOUSE = COMPUTE_WH
120 TARGET_LAG = '5 minutes'
121 AS
122 SELECT
123     COUNT(*) AS order_volume,
124     SUM(quantity) AS total_sales_quantity
125 FROM retail.gold.sales_fact;
126
127 SELECT * FROM retail.gold.dt_total_sales;
128
129
130 CREATE OR REPLACE DYNAMIC TABLE retail.gold.dt_top_products
131 WAREHOUSE = COMPUTE_WH
132 TARGET_LAG = '5 minutes'
133 AS
134 SELECT
135     p.product_name,
136     p.category,
137     SUM(f.quantity) AS total_quantity
138 FROM retail.gold.sales_fact f
139 JOIN retail.gold.product_dim p
140     ON f.product_key = p.product_key
141 GROUP BY p.product_name, p.category;
142
143 SELECT
144     product_name,
145     category,
146     total_quantity
147 FROM retail.gold.dt_top_products
148 ORDER BY total_quantity DESC
149 Limit 10;
150

```

```

151 CREATE OR REPLACE DYNAMIC TABLE retail.gold.dt_customer_behavior
152 WAREHOUSE = COMPUTE_WH
153 TARGET_LAG = '5 minutes'
154 AS
155 SELECT
156     c.customer_id,
157     COUNT(*) AS total_orders,
158     SUM(f.quantity) AS total_quantity
159 FROM retail.gold.sales_fact f
160 JOIN retail.gold.customer_dim c
161     ON f.customer_key = c.customer_key
162 GROUP BY c.customer_id;
163
164 SELECT
165     customer_id,
166     total_orders
167 FROM retail.gold.dt_customer_behavior
168 WHERE total_orders > 1;
169
170
171 CREATE OR REPLACE DYNAMIC TABLE retail.gold.dt_sales_trends
172 WAREHOUSE = COMPUTE_WH
173 TARGET_LAG = '5 minutes'
174 AS
175 SELECT
176     d.year,
177     d.month,
178     COUNT(*) AS order_volume,
179     SUM(f.quantity) AS total_quantity,
180     AVG(f.quantity) AS avg_order_value
181 FROM retail.gold.sales_fact f
182 JOIN retail.gold.date_dim d
183     ON f.date_key = d.date_key
184 GROUP BY d.year, d.month;
185

```

```
---
186 SELECT
187     year,
188     month,
189     order_volume,
190     total_quantity,
191     avg_order_value
192 FROM retail.gold.dt_sales_trends
193 ORDER BY year, month;
194
195
196
197 CREATE OR REPLACE DYNAMIC TABLE retail.gold.dt_store_performance
198 WAREHOUSE = COMPUTE_WH
199 TARGET_LAG = '5 minutes'
200 AS
201 SELECT
202     s.store_name,
203     SUM(if.quantity) AS total_quantity
204 FROM retail.gold.sales_fact f
205 JOIN retail.gold.store_dim s
206     ON f.store_key = s.store_key
207 GROUP BY s.store_name;
208
209
210 SELECT
211     customer_id,
212     total_orders
213 FROM retail.gold.dt_customer_behavior
214 WHERE total_orders > 1;
```